

Lesson Notes

MODULE 5 | LESSON 2

THE MONTY HALL PROBLEM

Reading Time	20 minutes
Prior Knowledge	Probability
Keywords	Factor, Factor graph

In the last lesson, we (re)introduced graph and network theory, contextualizing them in what we already know and also adding more rigorous theoretical background and nomenclature. We also refreshed some of our basic linear algebra and probability. Most importantly, we first introduced the Bayesian network. In this lesson, we look at ways to solve real-world problems like game show contests, which—however artificial the rules—still offer valuable insights into probability problems and the tools to solve them.

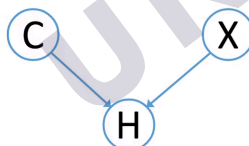
1. The Monty Hall Problem

As mentioned above, in this lesson, we will work to understand the Monty Hall problem, which can be quite difficult, even annoying, until your own "aha!" moment of clarity. Despite the difficulties it poses for the uninitiated, it lends itself to modeling as a Bayesian network due, ironically, to its structural simplicity and elegance. As you can see in figure 1, there are only three nodes and two arcs in this trivial seeming network.

Figure 1: The Monty Hall Problem as a Bayesian Network

Monty Hall Problem: Bayesian Network

X – Prize
C – Contestant
H – Host



$$P(C, X, H) = P(C)P(X)P(H|C, X)$$

Challenge: Find $P(X|C = c, H = h)$



Source: Galbraith, Byron. "Intro to Bayesian Networks: The Monty Hall Problem." *GitHub*. Slide 26. <https://github.com/bgalbraith/intro-to-bayesian-networks/blob/master/Boston%20Bayesians%20-%20Intro%20To%20Bayesian%20Networks.pdf>.

In this lesson, we review and run the code that uses (very simple) Bayesian networks to solve the Monty Hall problem (which is not very simple). In addition to the code, a [video is included as a required reading](#) because with the Monty Hall problem, understanding the correct answer and the reasoning behind it can require several and various explanations. With this in mind, the video supplements the other readings. However, this video was also chosen (from many available video explanations of the Monty Hall problem) for two additional reasons:

1. Meyer reminds the viewer about the important relationship between combinatorics (counting or enumeration, with factorials) and probability. This is good because these principles and this important relationship play a bigger role in the next lesson's reading. Pay attention to the first part of the video so you will be warmed up for the more sophisticated familiarity that is assumed in future lessons. We will, for example, be using combinatorics to calculate the number and probability of network structures based on the number of possible node orderings.
2. Meyer introduces a "tree model" for clarifying and solving the Monty Hall problem. We have seen many trees already over the courses of this program. The binomial option pricing model from Financial Markets looks very much like a tree, but is technically a lattice given the recombination that occurs between time steps. Of course, you recall the decision trees and random forests from the Machine Learning course. You also recall the Minimum Spanning Tree (and Kruskal's algorithm for

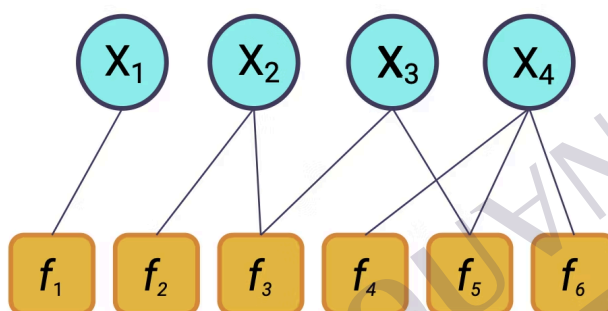
finding it) from Portfolio Management, which you will also be glad to know for the next module, where we learn *how to learn* the structure of a network. Finally, in this module, you have already seen basic graphs and network structures that appear tree-like, and you will see many more tree-like Bayesian networks; indeed, in the next module, you will read that acyclic graphs are referred to simply as trees. We will not spend much time on Meyer's tree model because this probability tree model actually does not relate to Bayesian networks (our real focus) in a direct meaningful way. So the inclusion of *yet another tree* is not meant to distract or confuse you; it is meant to increase your fluency with this simple but powerful structure as a general tool, which is only vaguely or abstractly related to the other contexts you see it in.

You will notice that in the first "pomegranate" required reading, Schreiber refers to **factor graphs** in the context of inference. So what is a factor graph? First, a **factor** in this context refers to a real-valued function of some subset of the random variables (Galbraith). More specifically, we can say with Mackay that in the following equation, the function $P(\mathbf{x})$ is the posterior probability distribution of N random variables, and this distribution function is defined as the product of the M factors $f_i(x_i)$, normalized by a constant Z .

$$P(\mathbf{x}) \equiv \frac{1}{Z} P^*(\mathbf{x}) = \frac{1}{Z} \prod_{m=1}^M f_m(x_m).$$

In a **factor graph**, the factors are often represented by squares and are connected by an arc to the variables (of which they are a function), represented by circles, as in figure 2.

Figure 2: A Simple Factor Graph



In the code comments in the module `FactorGraph.pyx`, Schreiber actually defines the factor graph slightly differently, as a "bipartite graph where conditional probability tables are on one side, and marginals for each of the variables involved are on the other side, but the gist is the same: the factors from our definition (above) encode the conditional probability tables he mentions, and he assigns the marginal probabilities directly to the variables themselves.

This is similar to the comment in the Pomegranate module `BayesianNetwork.pyx` that the `bake` function "includes converting conditional probability tables into joint probability tables and creating a list of both marginal and table nodes" (Schreiber).

To better understand factor graphs and their intrinsic relationship to Bayesian networks, read through the relevant code in the three links:

- <https://github.com/eBay/bayesian-belief-networks/blob/23c3e3ce1c7f99d83efb79f9b7798af2a1537e1c/bayesian/bbn.pyx>
- <https://github.com/jmschrei/pomegranate/blob/f115a242a5b50854bbf199d43fe2cfd061e9708a/pomegranate/FactorGraph.pyx>
- <https://github.com/jmschrei/pomegranate/blob/f115a242a5b50854bbf199d43fe2cfd061e9708a/pomegranate/BayesianNetworkWork.pyx>

They are not considered required readings, but the more familiar you are with this code can only help you later as we get deeper and look at code that is more demanding.

2. Conclusion

In this lesson, we took our first steps toward applying what we know about Graph and Network Theory to inference problems, like those that we might face as risk managers. The example we studied in this lesson, however, was very simple. In the next lesson, we look more closely at a slightly more complicated—and realistic—example, through the lens of Bayesian networks.

References

- eBay. "Bayesian Belief Networks Tutorial." *GitHub*. <https://github.com/eBay/bayesian-belief-networks>.
- Galbraith, Byron. "Intro to Bayesian Networks: The Monty Hall Problem." *GitHub*. <https://github.com/bgalbraith/intro-to-bayesian-networks/blob/master/Boston%20Bayesians%20-%20Intro%20To%20Bayesian%20Networks.pdf>.
- Mackay, David. "Exact Marginalization in Graphs." *Inference*. <http://www.inference.org.uk/mackay/itprnn/ps/334.340.pdf>.

- Meyer, Albert. "4.1.1 Tree Model: Video." *YouTube*. <https://youtu.be/dEsFEK4vnV4>.
- Schreiber, Jacob. "Pomegranate." *GitHub*. <https://github.com/jmschrei/pomegranate>.

